MODUL PRAKTIKUM
PEMROGRAMAN PERANGKAT BERGERAK

Materi	**Layout, State & SharedPreferences dengan UI Jetpack Compose**
Perangkat	Android Studio dengan Kotlin (Jetpack Compose)
Studi Kasus	Aplikasi Counter (Tambah, Kurang, Reset)
Hasil akhir	Memahami dan menerapkan Layout Row, Column, dan Box, State, serta SharedPreferences di ViewModel pada aplikasi Counter

1. Penjelasan Umum Jetpack Compose dan Layout

Jetpack Compose adalah toolkit modern dari Google untuk membangun UI Android secara **declarative**. Jika pada modul sebelumnya tampilan dibuat dengan file XML (LinearLayout, RelativeLayout) lalu dihubungkan ke Kotlin melalui View Binding, pada Compose seluruh tampilan ditulis langsung dengan kode Kotlin. Setiap komponen UI adalah sebuah fungsi yang diberi anotasi `@Composable`, misalnya `Text()`, `Button()`, dan `Icon()`.

Untuk mengatur posisi komponen, Compose menyediakan tiga layout dasar:

1. **Column –** Menyusun elemen secara vertikal (atas ke bawah). Padanannya di sistem View adalah LinearLayout dengan orientation vertical.
2. **Row –** Menyusun elemen secara horizontal (kiri ke kanan). Padanannya adalah LinearLayout dengan orientation horizontal.
3. **Box –** Menumpuk elemen di atas elemen lainnya (seperti layer). Elemen yang ditulis terakhir berada paling atas. Padanannya adalah FrameLayout.



Gambar 1. *Ilustrasi cara Column, Row, dan Box mengatur elemen*

Ketiga layout di atas dapat disusun bersarang (**nested**). Misalnya sebuah Column dapat berisi Row, dan Row dapat berisi Box. Tampilan, ukuran, dan perilaku setiap elemen diatur menggunakan **Modifier**, yaitu objek yang dirangkai (chaining) seperti `Modifier.fillMaxWidth().padding(16.dp)`. Urutan Modifier berpengaruh terhadap hasil akhir.

Parameter dan Modifier penting pada Layout:

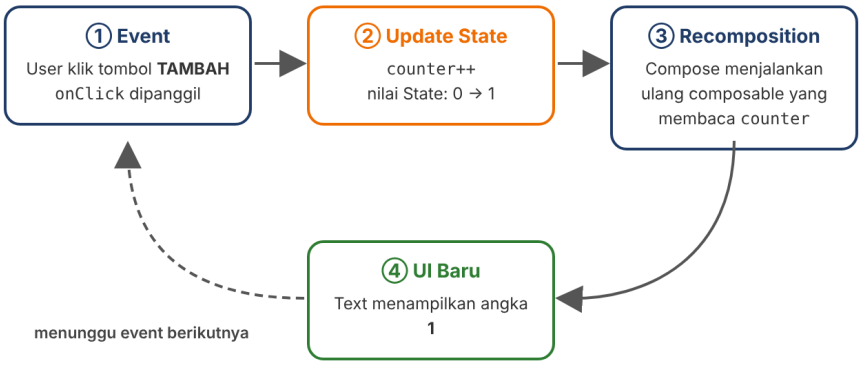
Parameter / Modifier	Dipakai pada	Fungsi
verticalArrangement	Column	Mengatur susunan/jarak anak secara vertikal, mis. Arrangement.spacedBy(8.dp)
horizontalAlignment	Column	Meratakan anak secara horizontal, mis. Alignment.CenterHorizontally
horizontalArrangement	Row	Mengatur susunan/jarak anak secara horizontal
verticalAlignment	Row	Meratakan anak secara vertikal, mis. Alignment.CenterVertically
contentAlignment	Box	Posisi default semua anak di dalam Box, mis. Alignment.Center
Modifier.align()	Anak dari Box	Mengatur posisi satu anak secara khusus, mis. Alignment.TopCenter
Modifier.weight()	Anak dari Row/Column	Membagi sisa ruang secara proporsional (seperti layout_weight)
fillMaxSize / fillMaxWidth	Semua composable	Mengisi seluruh ruang / seluruh lebar parent
padding / background	Semua composable	Memberi jarak dalam dan warna/bentuk latar belakang

Tabel Ringkasan Tipe Layout:

Layout	Cara Mengatur	Padanan di XML	Kegunaan Utama
Column	Berurutan vertikal	LinearLayout (vertical)	Form, daftar elemen, susunan halaman
Row	Berurutan horizontal	LinearLayout (horizontal)	Deretan tombol, ikon + teks
Box	Menumpuk (overlay)	FrameLayout	Badge, teks di atas gambar, posisi bebas di dalam area

2. Konsep State, ViewModel, dan SharedPreferences

State adalah setiap nilai yang dapat berubah seiring waktu dan memengaruhi tampilan. Pada aplikasi Counter, nilai angka counter adalah state. Di Compose, state dibuat dengan `mutableStateOf()` atau versi khusus Int `mutableIntStateOf()`. Ketika nilai state berubah, Compose otomatis menjalankan ulang (**recomposition**) fungsi composable yang membaca state tersebut sehingga UI selalu sesuai data. Kita tidak lagi memanggil `setText()` secara manual seperti pada View.



Gambar 2. *Siklus Event – State – Recomposition pada Jetpack Compose*

- **remember** – menyimpan state selama composable masih tampil (bertahan saat recomposition), tetapi hilang ketika Activity dibuat ulang, misalnya saat rotasi layar.
- **rememberSaveable** – seperti remember, tetapi nilainya ikut disimpan ke saved instance state sehingga bertahan saat rotasi layar.
- **ViewModel** – class dari Android Architecture Components untuk menyimpan state dan logika sebuah layar. ViewModel tetap hidup saat terjadi configuration change (rotasi), sehingga UI cukup menampilkan data dan meneruskan event.
- **SharedPreferences** – penyimpanan data sederhana berbentuk pasangan **key-value** yang ditulis ke file XML di internal storage aplikasi. Data tetap ada walaupun aplikasi ditutup atau HP di-restart.

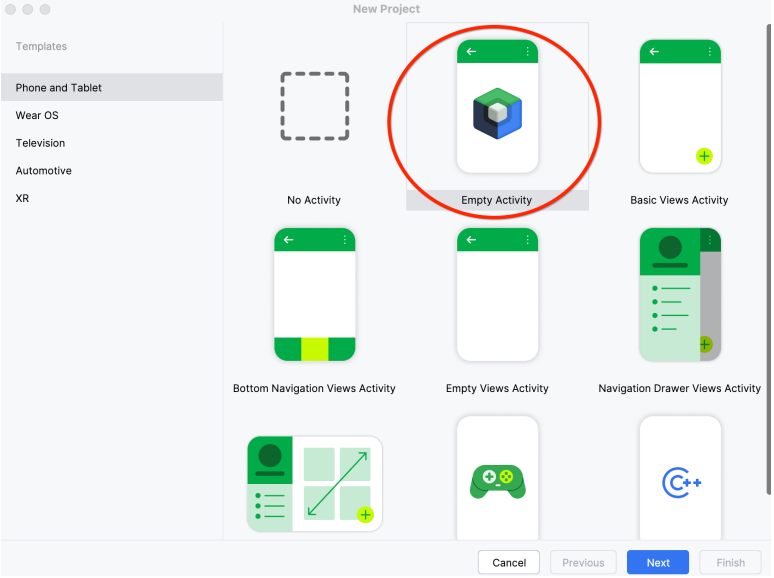
Pada modul ini, ketiga konsep tersebut digabungkan: UI dibangun dengan Column, Row, dan Box; nilai counter disimpan sebagai State di dalam ViewModel; dan setiap perubahan nilai disimpan ke SharedPreferences agar tidak hilang.

3. Codelab 1 – Layout Row, Column, dan Box pada Aplikasi Counter

Pada codelab ini kita membuat tampilan aplikasi Counter: judul di bagian atas, area angka yang besar di tengah, dan tombol KURANG, TAMBAH, serta RESET di bagian bawah. Pada tahap ini tampilan masih statis (angka belum bisa berubah).

Langkah 1: Membuat Project Baru

Buka Android Studio, pilih **File > New > New Project**, lalu pada kategori **Phone and Tablet** pilih template **Empty Activity** (template dengan Jetpack Compose, bukan Empty Views Activity). Isi konfigurasi berikut lalu klik **Finish**:



Konfigurasi	Nilai
Name	Counter Compose
Minimum SDK	API 24 ("Nougat"; Android 7.0)
Build configuration language	Kotlin DSL (build.gradle.kts)

Tunggu hingga proses Gradle Sync selesai. Struktur project dan file yang akan kita buat pada modul ini ditunjukkan pada gambar berikut.

```
Project ▾ (tampilan Android)
📁 app
 ├─ 📁 manifests
 ├─ 📁 kotlin+java
 │   └─ 📁 com.example.countercompose
 │       ├─ 📁 ui.theme
 │       ├─ 📄 MainActivity.kt
 │       ├─ 📄 CounterScreen.kt        ← Codelab 1 & 2
 │       └─ 📄 CounterViewModel.kt     ← Codelab 3
 └─ 📁 res
     └─ 📁 drawable
         ├─ 📄 ic_add.xml
         ├─ 📄 ic_remove.xml           ← Asset Studio
         └─ 📄 ic_refresh.xml
📁 Gradle Scripts
 ├─ 📄 build.gradle.kts (Module :app)   ← dependency
 └─ 📄 libs.versions.toml               ← version catalog
Kuning = file yang dibuat/ditambahkan pada modul ini
```

Gambar 3. *Struktur project Counter Compose (tampilan Android)*

Langkah 2: Menambahkan Icon dari Asset Studio

Seperti pada modul sebelumnya, kita menggunakan icon bawaan dari Asset Studio. Icon berupa vector drawable dapat dipakai di Compose melalui `painterResource()`.

1. Klik kanan pada folder **res** di panel Project, pilih **New > Vector Asset**.
2. Pada jendela Asset Studio, pilih **Clip Art** lalu klik ikon yang ada.
3. Cari icon **"add"**, pilih, lalu ubah nama menjadi `ic_add`. Klik **Next** lalu **Finish**.
4. Ulangi untuk icon **"remove"** dengan nama `ic_remove` dan icon **"refresh"** dengan nama `ic_refresh`.

Ketiga icon akan tersimpan di folder **res/drawable**. Warna icon tidak perlu diubah karena di dalam Button warna icon otomatis mengikuti warna konten tombol (putih).

Langkah 3: Membuat File CounterScreen.kt dengan Column

Klik kanan package **com.example.countercompose**, pilih **New > Kotlin Class/File**, pilih **File** dan beri nama **CounterScreen**. Tuliskan kode berikut. Import di bawah ini sudah mencakup seluruh kebutuhan Codelab 1.

```
package com.example.countercompose

import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.countercompose.ui.theme.CounterComposeTheme

@Composable
fun CounterScreen(modifier: Modifier = Modifier) {
    Column(
        modifier = modifier
            .fillMaxSize()
            .padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(
            text = "Counter App",
            fontSize = 24.sp,
            fontWeight = FontWeight.Bold
        )
        Spacer(modifier = Modifier.height(16.dp))
        CounterDisplay(
            value = 0,
            modifier = Modifier
                .fillMaxWidth()
                .weight(1f)
        )
        Spacer(modifier = Modifier.height(16.dp))
        CounterButtons()
    }
}
```

Penjelasan kode:

- `Column` menjadi wadah utama yang menyusun judul, area angka, dan tombol dari atas ke bawah.
- `horizontalAlignment = Alignment.CenterHorizontally` membuat semua anak Column rata tengah secara horizontal.
- `Spacer` memberi jarak kosong 16dp antar elemen.
- `Modifier.weight(1f)` pada CounterDisplay membuat area angka mengambil seluruh sisa tinggi layar, sama seperti `layout_weight="1"` pada LinearLayout.
- Parameter `modifier` diteruskan dari luar agar pemanggil (MainActivity) dapat menambahkan padding dari Scaffold.

Langkah 4: Membuat Area Angka dengan Box

Masih di file **CounterScreen.kt**, tambahkan fungsi `CounterDisplay` di bawah fungsi `CounterScreen`:

```
@Composable
fun CounterDisplay(value: Int, modifier: Modifier = Modifier) {
    Box(
        modifier = modifier.background(
            color = Color(0xFFF5F5F5),
            shape = RoundedCornerShape(16.dp)
        ),
        contentAlignment = Alignment.Center
    ) {
        Text(
            text = "Nilai Counter",
            fontSize = 14.sp,
            color = Color.Gray,
            modifier = Modifier
                .align(Alignment.TopCenter)
                .padding(top = 16.dp)
        )
        Text(
            text = value.toString(),
            fontSize = 120.sp,
            fontWeight = FontWeight.Bold
        )
    }
}
```

Penjelasan kode:

- `Box` menumpuk dua Text di area yang sama, dengan latar abu-abu bersudut membulat.
- `contentAlignment = Alignment.Center` membuat angka counter berada tepat di tengah Box.
- Text "Nilai Counter" menggunakan `Modifier.align(Alignment.TopCenter)` sehingga posisinya berbeda dari default, yaitu di tengah atas Box. Inilah keunggulan Box: setiap anak dapat ditempatkan bebas di dalam area yang sama.

Langkah 5: Membuat Deretan Tombol dengan Row

Tambahkan fungsi `CounterButtons` di bawah `CounterDisplay`. Parameter berupa lambda `() -> Unit` disiapkan sekarang agar nanti tombol mudah dihubungkan dengan logika.

```
@Composable
fun CounterButtons(
    onDecrement: () -> Unit = {},
    onIncrement: () -> Unit = {},
    onReset: () -> Unit = {}
) {
    Column(modifier = Modifier.fillMaxWidth()) {
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.spacedBy(8.dp)
        ) {
            Button(
                onClick = onDecrement,
                modifier = Modifier.weight(1f),
                colors = ButtonDefaults.buttonColors(
                    containerColor = Color(0xFFF44336))
            ) {
                Icon(
                    painter = painterResource(R.drawable.ic_remove),
                    contentDescription = null,
                    modifier = Modifier.size(ButtonDefaults.IconSize)
                )
                Spacer(Modifier.width(ButtonDefaults.IconSpacing))
                Text("KURANG")
            }
            Button(
                onClick = onIncrement,
                modifier = Modifier.weight(1f),
                colors = ButtonDefaults.buttonColors(
                    containerColor = Color(0xFF4CAF50))
            ) {
                Icon(
                    painter = painterResource(R.drawable.ic_add),
                    contentDescription = null,
                    modifier = Modifier.size(ButtonDefaults.IconSize)
                )
                Spacer(Modifier.width(ButtonDefaults.IconSpacing))
                Text("TAMBAH")
            }
        }
        Spacer(modifier = Modifier.height(8.dp))
        Button(
            onClick = onReset,
            modifier = Modifier.fillMaxWidth(),
            colors = ButtonDefaults.buttonColors(
                containerColor = Color(0xFF2196F3))
        ) {
            Icon(
                painter = painterResource(R.drawable.ic_refresh),
                contentDescription = null,
                modifier = Modifier.size(ButtonDefaults.IconSize)
            )
            Spacer(Modifier.width(ButtonDefaults.IconSpacing))
            Text("RESET")
        }
    }
}
```

Penjelasan kode:

- `Row` menyusun tombol KURANG dan TAMBAH berdampingan dari kiri ke kanan.
- `Modifier.weight(1f)` pada kedua tombol membuat lebarnya sama besar; `Arrangement.spacedBy(8.dp)` memberi jarak 8dp di antaranya.
- Isi Button sendiri juga disusun secara horizontal (Button memiliki RowScope), sehingga Icon, Spacer, dan Text tampil berderet.
- Tombol RESET berada di luar Row dengan `fillMaxWidth()` sehingga memenuhi lebar layar.

Langkah 6: Menambahkan Preview dan Memanggil dari MainActivity

Tambahkan fungsi Preview di bagian paling bawah **CounterScreen.kt** agar tampilan dapat dilihat langsung di panel **Split/Design** tanpa menjalankan emulator:

```
@Preview(showBackground = true, showSystemUi = true)
@Composable
fun CounterScreenPreview() {
    CounterComposeTheme {
        CounterScreen()
    }
}
```

Kemudian buka **MainActivity.kt**. Hapus fungsi `Greeting` dan `GreetingPreview` bawaan template, lalu sesuaikan kodenya:

```
package com.example.countercompose

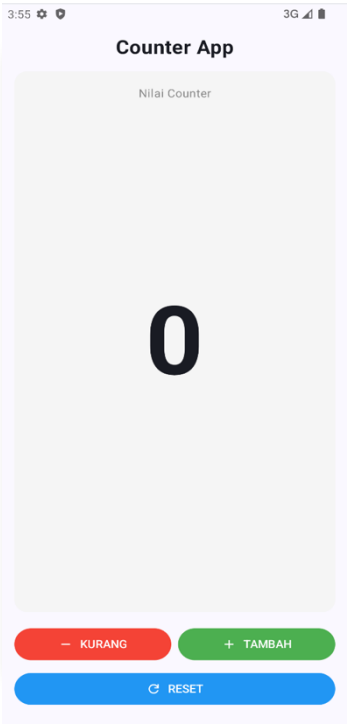
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Scaffold
import androidx.compose.ui.Modifier
import com.example.countercompose.ui.theme.CounterComposeTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            CounterComposeTheme {
                Scaffold(
                    modifier = Modifier.fillMaxSize()
                ) { innerPadding ->
                    CounterScreen(
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }
}
```

`setContent { }` menggantikan `setContentView()` pada View. `Scaffold` menyediakan kerangka layar Material 3, dan `innerPadding` memastikan konten tidak tertutup status bar maupun navigation bar.

Langkah 7: Menjalankan Aplikasi

Jalankan project dengan menekan tombol **Run** atau **Shift+F10**. Aplikasi akan tampil seperti gambar berikut. Garis putus-putus menunjukkan bagian layar yang dibangun oleh Column, Box, dan Row. Perhatikan bahwa ketika tombol diklik, angka **belum berubah** karena kita belum menggunakan State.




```
Column  ← wadah utama (CounterScreen)
 ├ Text            "Counter App"
 ├ Box   ← CounterDisplay (weight 1f)
 │   ├ Text        "Nilai Counter" (TopCenter)
 │   └ Text        angka counter (Center)
 └ Column          ← CounterButtons
     ├ Row
     │   ├ Button  KURANG (weight 1f)
     │   └ Button  TAMBAH (weight 1f)
     └ Button      RESET (fillMaxWidth)
```

Gambar 4. *Hasil Codelab 1 dan pemetaan Column, Box, serta Row pada layar*

4. Codelab 2 – Implementasi State pada Counter

Pada codelab ini kita membuat angka counter dapat berubah ketika tombol diklik dengan memanfaatkan State dan recomposition.

Langkah 1: Menambahkan State dengan remember

Buka **CounterScreen.kt** dan tambahkan import berikut di bagian atas file:

```
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableIntStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
```

Lalu ubah fungsi `CounterScreen` menjadi seperti berikut (bagian yang berubah adalah deklarasi state, parameter `value`, dan pemanggilan `CounterButtons`):

```
@Composable
fun CounterScreen(modifier: Modifier = Modifier) {
    var counter by remember { mutableIntStateOf(0) }

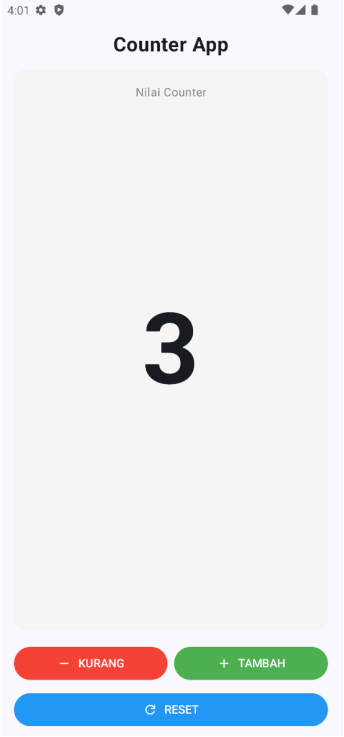
    Column(
        modifier = modifier
            .fillMaxSize()
            .padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(
            text = "Counter App",
            fontSize = 24.sp,
            fontWeight = FontWeight.Bold
        )
        Spacer(modifier = Modifier.height(16.dp))
        CounterDisplay(
            value = counter,
            modifier = Modifier.fillMaxWidth().weight(1f)
        )
        Spacer(modifier = Modifier.height(16.dp))
        CounterButtons(
            onDecrement = { counter-- },
            onIncrement = { counter++ },
            onReset = { counter = 0 }
        )
    }
}
```

Penjelasan kode:

- `mutableIntStateOf(0)` membuat state bertipe Int dengan nilai awal 0. Compose "mengamati" state ini.
- `remember { }` menjaga agar state tidak dibuat ulang menjadi 0 setiap kali terjadi recomposition.
- Kata kunci `by` (property delegate) membuat kita bisa menulis `counter++` secara langsung tanpa `.intValue`. Untuk itu diperlukan import `getValue` dan `setValue`.
- Setiap klik tombol mengubah nilai `counter`, lalu Compose melakukan recomposition pada `CounterDisplay` sehingga angka di layar ikut berubah.

Langkah 2: Menjalankan dan Menguji State

Jalankan aplikasi, lalu coba klik tombol TAMBAH beberapa kali, KURANG, dan RESET. Angka sekarang berubah sesuai tombol yang diklik.



Gambar 5. *Hasil Codelab 2: angka berubah setiap tombol diklik*

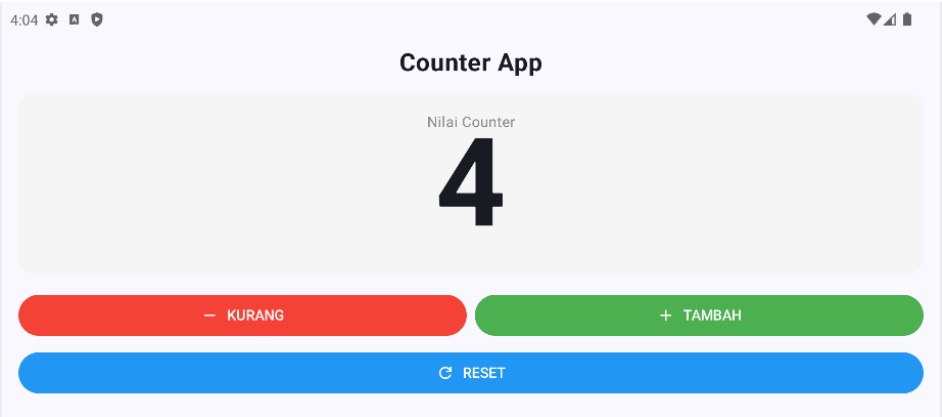
Langkah 3: Menguji Rotasi Layar dan Menggunakan rememberSaveable

Aktifkan **Auto-rotate** pada emulator, naikkan nilai counter menjadi 5, lalu putar layar ke posisi landscape. Nilai counter akan kembali ke **0**. Hal ini terjadi karena rotasi adalah **configuration change** yang membuat Activity dihancurkan dan dibuat ulang, sehingga state dari `remember` ikut hilang.

Untuk mengatasinya, ganti `remember` dengan `rememberSaveable`:

```
import androidx.compose.runtime.saveable.rememberSaveable

// di dalam CounterScreen
var counter by rememberSaveable { mutableIntStateOf(0) }
```

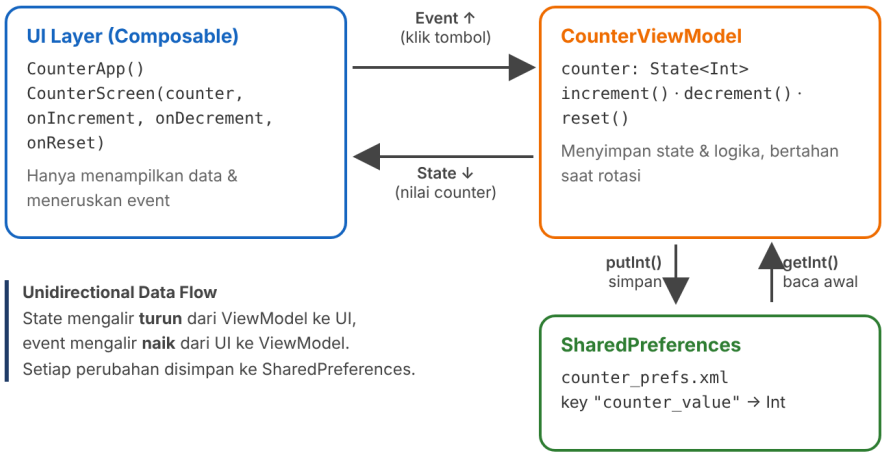


Gambar 6. *Perbedaan remember dan rememberSaveable saat rotasi layar*

Jalankan kembali dan ulangi pengujian rotasi; nilai counter sekarang tetap 5. Namun, coba **tutup aplikasi** dari Recents lalu buka kembali: nilai kembali 0. Selain itu, logika counter masih bercampur dengan kode UI. Kedua masalah ini akan kita selesaikan pada Codelab 3 dengan ViewModel dan SharedPreferences.

5. Codelab 3 – SharedPreferences di ViewModel

Pada codelab ini nilai counter dipindahkan ke dalam **ViewModel**, dan setiap perubahan nilai disimpan ke **SharedPreferences**. UI hanya bertugas menampilkan nilai dan meneruskan event klik ke ViewModel. Pola ini disebut **Unidirectional Data Flow (UDF)**.



Gambar 7. *Arsitektur aplikasi: UI – ViewModel – SharedPreferences*

Langkah 1: Menambahkan Dependency ViewModel untuk Compose

Buka **gradle/libs.versions.toml**. Pada bagian `[libraries]`, tambahkan baris berikut. Versinya mengikuti `lifecycleRuntimeKtx` yang sudah ada dari template, dan entri library harus ditulis dalam **satu baris**:

```
[libraries]
# ... library bawaan template
androidx-lifecycle-viewmodel-compose = { group = "androidx.lifecycle", name =
"lifecycle-viewmodel-compose", version.ref = "lifecycleRuntimeKtx" }
```

Kemudian buka **build.gradle.kts (Module :app)** dan tambahkan pada blok `dependencies`:

```
dependencies {
    // ... dependency bawaan template
    implementation(libs.androidx.lifecycle.viewmodel.compose)
}
```

Klik **Sync Now** setelah perubahan Gradle. Library ini menyediakan fungsi `viewModel()` untuk mengambil ViewModel dari dalam composable.

Langkah 2: Membuat CounterViewModel dengan SharedPreferences

Klik kanan package **com.example.countercompose > New > Kotlin Class/File**, pilih **Class**, beri nama **CounterViewModel**, lalu tuliskan kode berikut:

```
package com.example.countercompose

import android.app.Application
import android.content.Context
import androidx.compose.runtime.State
import androidx.compose.runtime.mutableIntStateOf
import androidx.core.content.edit
import androidx.lifecycle.AndroidViewModel

class CounterViewModel(
    application: Application
) : AndroidViewModel(application) {

    // 1. Membuka/membuat file SharedPreferences
    private val prefs = application.getSharedPreferences(
        PREFS_NAME, Context.MODE_PRIVATE
    )

    // 2. State awal dibaca dari SharedPreferences (default 0)
    private val _counter = mutableIntStateOf(
        prefs.getInt(KEY_COUNTER, 0)
    )
    val counter: State<Int> = _counter

    // 3. Fungsi yang dipanggil dari UI
    fun increment() = updateCounter(_counter.intValue + 1)
    fun decrement() = updateCounter(_counter.intValue - 1)
    fun reset() = updateCounter(0)

    // 4. Ubah state lalu simpan ke SharedPreferences
    private fun updateCounter(newValue: Int) {
        _counter.intValue = newValue
        prefs.edit { putInt(KEY_COUNTER, newValue) }
    }

    companion object {
        private const val PREFS_NAME = "counter_prefs"
        private const val KEY_COUNTER = "counter_value"
    }
}
```

Penjelasan kode:

- `AndroidViewModel` adalah turunan ViewModel yang menerima objek `Application`. Application dibutuhkan sebagai **Context** untuk membuka SharedPreferences. Jangan menyimpan Context milik Activity di ViewModel karena dapat menyebabkan memory leak.
- `getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE)` membuka file preferences yang hanya dapat diakses oleh aplikasi ini.
- `prefs.getInt(KEY_COUNTER, 0)` membaca nilai tersimpan saat ViewModel pertama kali dibuat; jika belum ada, nilai default 0.

- `_counter` bersifat **private** dan **mutable**, sedangkan `counter` bersifat **public** dan **read-only** (`State<Int>`). Dengan begitu, UI hanya bisa membaca nilai dan perubahan hanya bisa dilakukan lewat fungsi ViewModel (**encapsulation**).
- `prefs.edit { putInt(...) }` adalah extension function dari core-ktx yang menulis data lalu otomatis memanggil `apply()` (penyimpanan asynchronous).

Langkah 3: State Hoisting pada CounterScreen

Agar `CounterScreen` tidak lagi menyimpan state sendiri, kita terapkan **State Hoisting**: state "diangkat" ke pemanggilnya. `CounterScreen` menjadi **stateless** (hanya menerima nilai dan lambda), sedangkan fungsi baru `CounterApp` menghubungkannya dengan ViewModel. Tambahkan import berikut di **CounterScreen.kt**:

```
import androidx.lifecycle.viewmodel.compose.viewModel
```

Kemudian ganti fungsi `CounterScreen` dan `CounterScreenPreview` dengan kode berikut (fungsi `CounterDisplay` dan `CounterButtons` tidak berubah). Import `remember`, `rememberSaveable`, `mutableIntStateOf`, dan `setValue` yang tidak terpakai boleh dihapus, tetapi `getValue` tetap diperlukan.

```
@Composable
fun CounterApp(
    modifier: Modifier = Modifier,
    viewModel: CounterViewModel = viewModel()
) {
    val counter by viewModel.counter

    CounterScreen(
        counter = counter,
        onIncrement = { viewModel.increment() },
        onDecrement = { viewModel.decrement() },
        onReset = { viewModel.reset() },
        modifier = modifier
    )
}

@Composable
fun CounterScreen(
    counter: Int,
    onIncrement: () -> Unit,
    onDecrement: () -> Unit,
    onReset: () -> Unit,
    modifier: Modifier = Modifier
) {
    Column(
        modifier = modifier
            .fillMaxSize()
            .padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(
            text = "Counter App",
            fontSize = 24.sp,
            fontWeight = FontWeight.Bold
        )
        Spacer(modifier = Modifier.height(16.dp))
        CounterDisplay(
```

```
            value = counter,
            modifier = Modifier.fillMaxWidth().weight(1f)
        )
        Spacer(modifier = Modifier.height(16.dp))
        CounterButtons(
            onDecrement = onDecrement,
            onIncrement = onIncrement,
            onReset = onReset
        )
    }
}

@Preview(showBackground = true, showSystemUi = true)
@Composable
fun CounterScreenPreview() {
    CounterComposeTheme {
        CounterScreen(
            counter = 7,
            onIncrement = {},
            onDecrement = {},
            onReset = {}
        )
    }
}
```

Penjelasan kode:

- `viewModel()` mengambil instance `CounterViewModel` yang terikat pada Activity. Saat layar dirotasi, instance yang sama dipakai kembali sehingga nilai tidak hilang.
- `val counter by viewModel.counter` membaca State dari ViewModel. Setiap kali nilainya berubah, `CounterApp` dan `CounterScreen` mengalami recomposition.
- Event klik diteruskan ke fungsi `increment()`, `decrement()`, dan `reset()` milik ViewModel (event naik), sedangkan nilai counter dikirim ke UI sebagai parameter (state turun).
- Karena `CounterScreen` stateless, Preview dapat menampilkan nilai apa pun (misalnya 7) tanpa perlu ViewModel.

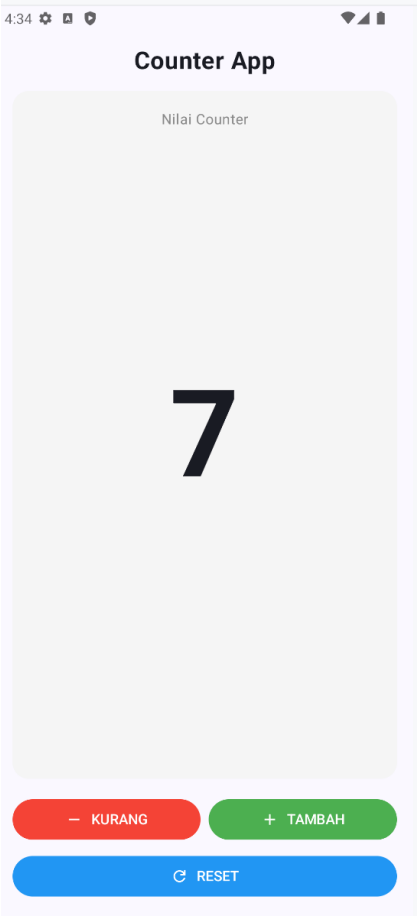
Langkah 4: Memanggil CounterApp dari MainActivity

Buka **MainActivity.kt** dan ganti pemanggilan `CounterScreen` di dalam Scaffold menjadi `CounterApp`:

```
Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
    CounterApp(modifier = Modifier.padding(innerPadding))
}
```

Langkah 5: Menjalankan dan Menguji Penyimpanan Data

Jalankan aplikasi dan lakukan pengujian berikut: (1) klik TAMBAH hingga angka 7, (2) tutup aplikasi sepenuhnya dengan swipe dari layar **Recents**, (3) buka kembali aplikasi. Angka tetap **7** karena nilai dibaca ulang dari SharedPreferences saat ViewModel dibuat. Uji juga rotasi layar; nilai tetap terjaga karena ViewModel bertahan saat configuration change.



Gambar 8. *Pengujian persistensi: nilai counter tetap tersimpan setelah aplikasi ditutup*

Langkah 6: Melihat Isi File SharedPreferences

Untuk memastikan data benar-benar tersimpan, buka **View > Tool Windows > Device Explorer** saat emulator berjalan, lalu masuk ke folder **/data/data/com.example.countercompose/shared_prefs/** dan klik dua kali file **counter_prefs.xml**. Isinya kurang lebih seperti berikut:

```
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <int name="counter_value" value="7" />
</map>
```

Terlihat bahwa SharedPreferences menyimpan data sebagai file XML berisi pasangan key (`counter_value`) dan value (`7`).

> **Catatan:** SharedPreferences cocok untuk data kecil dan sederhana seperti pengaturan atau nilai counter. Untuk data yang lebih besar atau terstruktur, gunakan Jetpack DataStore atau database Room.

6. Perbandingan Teknik Penyimpanan State

Aspek	remember	rememberSaveable	ViewModel	SharedPreferences
Bertahan saat recomposition	Ya	Ya	Ya	Ya
Bertahan saat rotasi layar	Tidak	Ya	Ya	Ya
Bertahan saat aplikasi ditutup	Tidak	Tidak	Tidak	Ya
Lokasi data	Memori (Composition)	Saved instance state (Bundle)	Memori (ViewModelStore)	File XML di internal storage
Kapan digunakan	State UI sementara, mis. buka/tutup menu	Input form, pilihan tab	State layar dan logika bisnis	Pengaturan dan data kecil yang permanen

Pada aplikasi Counter, kombinasi **ViewModel + SharedPreferences** memberikan hasil terbaik: ViewModel mengelola state dan logika selama aplikasi berjalan, sedangkan SharedPreferences menjamin nilai tetap ada saat aplikasi dibuka kembali.

8. Ringkasan

- Jetpack Compose membangun UI secara declarative menggunakan fungsi `@Composable`, tanpa file layout XML.
- **Column** menyusun elemen vertikal, **Row** menyusun elemen horizontal, dan **Box** menumpuk elemen dalam satu area.
- **Modifier** mengatur ukuran, jarak, latar, dan posisi elemen; `weight()` membagi ruang secara proporsional.
- **State** (`mutableIntStateOf`) memicu **recomposition** sehingga UI selalu sesuai data. `remember` hilang saat rotasi, `rememberSaveable` bertahan saat rotasi.
- **ViewModel** menyimpan state dan logika layar serta bertahan saat configuration change. **State Hoisting** membuat composable menjadi stateless dan mudah di-preview.
- **SharedPreferences** menyimpan data key-value secara permanen; dengan memanggilnya di ViewModel, nilai counter tetap tersimpan meskipun aplikasi ditutup.

Sumber Referensi

- Android Developers. Compose https://developer.android.com/compose